FLUTTER DO ZERO
AO FRONT-END COMPLETO

Guia autoinstrucional para consumir a API Gestão Comercial no VS Code

Clientes • Produtos • Serviços • Vendas

Apostila prática, guiada e comentada

Prof. José Luiz Lanzarini
UNIPAR - Universidade Paranaense

Apresentação

Esta edição foi elaborada para uma pessoa que **nunca programou em Flutter** e precisa desenvolver o projeto sem acompanhamento presencial. O estudante será conduzido desde a instalação e os conceitos mínimos de Dart até uma aplicação completa conectada à API REST de gestão comercial.

Nada deve ser presumido. Cada etapa informa onde clicar, qual pasta abrir, qual arquivo criar, o que digitar, por que o código existe, como executar e qual resultado deve aparecer. Os códigos completos continuam disponíveis para conferência, mas antes deles são explicados os blocos que os compõem.

Pré-requisito real: saber utilizar pastas, arquivos, navegador e copiar comandos para o terminal. Conhecimento anterior de Flutter não é necessário.

Como estudar sem professor

1. Não pule o Módulo Zero. Ele apresenta as palavras e símbolos usados no restante do material.
2. Execute um passo por vez. Não crie todos os arquivos de uma única vez.
3. Depois de cada checkpoint, execute `flutter analyze`. Continue somente quando o erro descrito estiver resolvido.
4. Digite inicialmente os pequenos exemplos. Nos arquivos extensos, copie o código completo e depois acompanhe a explicação de cada bloco.
5. Ao encontrar erro, leia a primeira mensagem vermelha, confira arquivo e número da linha e consulte a seção de diagnóstico.
6. Mantenha a API aberta em uma aba e o Flutter em outra para distinguir erro do front-end e erro do back-end.

Objetivos de aprendizagem

- Compreender o vocabulário básico de Dart e Flutter.
- Estruturar uma aplicação em camadas simples.
- Converter JSON em objetos Dart.
- Executar GET, POST, PUT, PATCH e DELETE.
- Construir formulários, listas e navegação responsiva.
- Tratar carregamento, lista vazia, sucesso e erro.
- Consumir uma API local ou hospedada no Render.

Sumário

Etapa	Conteúdo
0	Primeiros conceitos: programação, Dart e Flutter
1	Visão geral da solução
2	Instalação e preparação completa do Windows
3	Criação do projeto no VS Code
4	Dependências e organização de pastas
5	Configuração do endereço da API
6	Modelagem dos dados
7	Camada de comunicação HTTP
8	Widgets reutilizáveis
9	Aplicação, tema e navegação
10	CRUD de clientes
11	CRUD de produtos
12	CRUD de serviços
13	Cadastro e consulta de vendas
14	Execução, depuração e integração com Render
15	Testes, problemas comuns e roteiro autônomo

0. Módulo Zero: antes de programar

Este módulo apresenta somente o necessário para compreender o projeto. Leia mesmo que pretenda copiar o código, pois ele permitirá localizar e corrigir erros.

0.1 O que é programar?

Programar é escrever instruções que o computador executará. Essas instruções precisam respeitar a sintaxe de uma linguagem. No projeto, usamos Dart como linguagem e Flutter como conjunto de ferramentas para construir a interface.

Termo	Significado simples
Dart	Linguagem em que escrevemos as instruções
Flutter	Framework que fornece widgets e ferramentas
Widget	Peça da interface: texto, botão, campo ou tela
API	Serviço externo que recebe e devolve dados
JSON	Formato textual utilizado para transportar dados
Compilar	Transformar o projeto em uma aplicação executável
Terminal	Área do VS Code onde digitamos comandos

0.2 Como ler uma linha de Dart

```
final nome = 'Maria';
```

- `final`: a variável recebe o valor uma vez.
- `nome`: nome escolhido para a variável.
- `=`: atribui o valor da direita à variável.
- `'Maria'`: texto, chamado String.
- `;`: finaliza a instrução.

Experimento 1

Crie temporariamente um arquivo `teste.dart`, escreva as linhas abaixo e observe as cores atribuídas pelo VS Code:

```
void main() {
  final nome = 'Maria';
  final idade = 20;
  final ativo = true;
  print('$nome tem $idade anos. Ativo: $ativo');
}
```

Não é necessário manter esse arquivo no projeto final.

0.3 Tipos utilizados

Tipo	Guarda	Exemplo
String	Texto	'Cliente'
int	Número inteiro	15
double	Número decimal	89.90
bool	Verdadeiro ou falso	true
List	Vários elementos	List
Map	Pares de chave e valor	JSON
Tipo?	Valor que pode ser nulo	String? email

0.4 Funções

```
double calcularSubtotal(double preco, int quantidade) {
  return preco * quantidade;
}
```

Uma função possui nome, parâmetros, instruções e, em muitos casos, um retorno. No projeto, funções como `_carregar()` e `salvarCliente()` organizam as tarefas.

0.5 Classes e objetos

```
class Produto {
  final String nome;
  final double preco;

  Produto({required this.nome, required this.preco});
}

final mouse = Produto(nome: 'Mouse', preco: 59.90);
```

A classe é o molde; `mouse` é um objeto construído a partir desse molde. Os modelos do projeto seguem essa ideia.

0.6 Null Safety

```
String nome = 'Maria';       // não aceita nulo
String? telefone;           // pode ser nulo
final texto = telefone ?? 'Sem telefone';
```

O ponto de interrogação após o tipo indica que o valor pode não existir. O operador ?? fornece um valor alternativo.

0.7 Future, async e await

Requisições pela internet levam tempo. Flutter representa um resultado futuro com `Future`. A palavra `await` aguarda o resultado dentro de uma função `async`.

```
Future<void> carregar() async {
  final clientes = await api.listarClientes();
  print(clientes.length);
}
```

A interface não fica congelada enquanto a rede responde.

0.8 O que é um widget?

```
const Text('Olá, Flutter!')

FilledButton(
  onPressed: () {},
  child: const Text('Salvar'),
)
```

Widgets são combinados como uma árvore. Um botão possui um widget Text como filho. Parênteses agrupam parâmetros; chaves delimitam blocos; colchetes representam listas.

0.9 StatelessWidget e StatefulWidget

Tipo	Quando utilizar
StatelessWidget	Quando o widget apenas recebe e exibe informações
StatefulWidget	Quando os dados mudam e a tela precisa ser reconstruída

O método `setState` avisa ao Flutter que um valor mudou e que a interface deve ser reconstruída.

0.10 Símbolos que aparecerão

Símbolo	Leitura
()	Parâmetros ou chamada de função
{}	Bloco de código ou mapa
[]	Lista ou acesso por posição
.	Acesso a método ou propriedade
=>	Função curta que devolve uma expressão
!	Afirma que um valor não é nulo
?	Valor opcional/anulável
${...}	Inserção de valor dentro de texto

Checkpoint do Módulo Zero

Antes de continuar, o estudante deve conseguir explicar com palavras próprias: variável, função, classe, objeto, lista, widget, Future e JSON. Não é necessário memorizar a sintaxe.

1. Visão geral da solução

A aplicação Flutter não acessa diretamente o PostgreSQL. Toda comunicação ocorre por HTTP com a API Spring Boot. A API valida os dados, aplica regras de negócio, atualiza o estoque e persiste os registros.

Camada	Responsabilidade	Exemplo
Interface Flutter	Exibir dados e capturar ações	Formulário de cliente
Modelos Dart	Representar os objetos recebidos	Cliente, Produto e Venda
ApiService	Enviar requisições HTTP	GET /api/produtos
API Spring	Validar e executar regras	Calcular total da venda
PostgreSQL	Persistir os registros	Tabelas clientes e vendas

Fluxo de uma consulta

1. O usuário abre a aba Produtos.
2. A tela chama `ApiService.listarProdutos()`.
3. O serviço envia `GET /api/produtos`.
4. O Spring consulta o PostgreSQL e responde com um array JSON.
5. O Flutter converte cada objeto JSON em um `Produto`.
6. A tela atualiza a lista e apresenta os cartões.

Regra essencial

O front-end pode validar a experiência do usuário, mas a validação definitiva e as regras críticas ficam no back-end. Por exemplo, o Flutter envia cliente e itens da venda, mas o preço, o total e a redução do estoque são calculados pela API.

2. Preparação completa do Windows

Se o Flutter já funciona no computador, avance para a verificação. Caso contrário, siga a instalação inteira.

2.1 Instalar o Git

1. Abra o navegador e acesse `https://git-scm.com/download/win`.
2. Baixe o instalador de 64 bits.
3. Execute o instalador e mantenha as opções padrão.
4. Conclua e reinicie o VS Code caso ele estivesse aberto.
5. Abra o Prompt de Comando e execute `git --version`.

Resultado esperado:

```
git version 2.x.x
```

2.2 Baixar e extrair o Flutter SDK

1. Acesse `https://docs.flutter.dev/get-started/install/windows`.
2. Baixe o pacote estável do Flutter para Windows.
3. Crie a pasta `C:\src`.
4. Extraia o conteúdo de modo que exista `C:\src\flutter\bin`.
5. Não instale dentro de Arquivos de Programas, pois permissões podem causar erros.

A estrutura correta é:

```
C:\src\flutter\bin\flutter.bat
```

2.3 Adicionar Flutter ao PATH

1. Abra o menu Iniciar e pesquise **variáveis de ambiente**.
2. Selecione **Editar as variáveis de ambiente do sistema**.
3. Clique em **Variáveis de Ambiente**.
4. Em Variáveis do usuário, selecione **Path** e clique em **Editar**.
5. Clique em **Novo** e informe `C:\src\flutter\bin`.
6. Confirme todas as janelas com OK.
7. Feche e abra novamente o terminal.

Teste:

```
flutter --version
```

> Se “flutter não é reconhecido” aparecer, o terminal ainda não recebeu o PATH ou o caminho foi digitado incorretamente.

2.4 Instalar VS Code e Chrome

1. Instale o Google Chrome pelo site oficial.
2. Baixe o Visual Studio Code em `https://code.visualstudio.com`.
3. Durante a instalação, marque as opções para adicionar ao PATH e abrir pastas com o VS Code.
4. Inicie o VS Code.

2.5 Extensões do VS Code

1. Clique no ícone de quatro blocos na barra esquerda, chamado **Extensions**.
2. No campo de busca, digite `Flutter`.
3. Selecione a extensão publicada pela Dart Code e clique em **Install**.
4. A extensão Dart será instalada automaticamente.
5. Aguarde até os botões mostrarem “Disable” ou “Uninstall”, indicando instalação concluída.

2.6 Verificação completa

Abra **Terminal** → **New Terminal** no menu superior e execute:

```
flutter doctor -v
```

Para desenvolver no Chrome, as linhas Flutter, Chrome e VS Code precisam mostrar confirmação. Alertas apenas do Android Studio podem ser ignorados nesta atividade.

Se Flutter solicitar aceite de licença ou download de componentes, siga a instrução indicada no próprio terminal.

2.7 Ativar e localizar o Chrome

```
flutter config --enable-web
flutter devices
```

Resultado esperado: uma linha contendo `Chrome` e `web-javascript`.

2.8 Verificar a API antes do front-end

1. Abra o Chrome.
2. Cole a URL local `http://localhost:8080/swagger-ui.html` ou a URL do Render.
3. Confirme que a página Swagger carrega.
4. Abra `GET /api/clientes`, clique em **Try it out** e depois **Execute**.
5. A resposta deve ter status 200 e apresentar um array JSON.

> Se essa verificação falhar, resolva primeiro a API. O Flutter depende dela e não corrige falhas do back-end.

3. Criação do projeto Flutter

3.1 Escolher a pasta

1. No Windows, crie uma pasta chamada `ProjetosFlutter` dentro de Documentos.
2. Abra o VS Code.
3. Selecione **File** → **Open Folder**.
4. Escolha a pasta ProjetosFlutter e confirme em **Select Folder**.
5. Se o VS Code perguntar se você confia nos autores, confirme somente por ser uma pasta criada por você.

3.2 Abrir o terminal correto

1. No menu superior, clique em **Terminal**.
2. Clique em **New Terminal**.
3. Observe a linha de comando na parte inferior.
4. Confirme que o caminho termina em `ProjetosFlutter`.

3.3 Criar o projeto

```
flutter create flutter_gestao_comercial
```

Espere a mensagem de conclusão. Em seguida, entre na pasta e abra o projeto:

```
cd flutter_gestao_comercial
code .
```

Uma nova janela pode ser aberta. A raiz exibida no Explorer deve ser `flutter_gestao_comercial` e deve conter `lib` e `pubspec.yaml`.

3.4 Conhecer o Explorer

Item	Uso neste projeto
lib	Todo o código Dart desenvolvido
web	Arquivos necessários para executar no Chrome
test	Testes; não será o foco inicial
pubspec.yaml	Dependências e configuração
analysis_options.yaml	Regras de análise do código

3.5 Executar o projeto original

```
flutter pub get
flutter run -d chrome
```

1. Espere o Chrome abrir automaticamente.
2. A página padrão mostrará um contador e um botão “+”.
3. Volte ao terminal do VS Code.
4. Pressione q para encerrar.

> Se o Chrome abrir uma página em branco, espere o terminal concluir a compilação. Se houver texto vermelho, leia a primeira mensagem e não as dezenas de mensagens posteriores.

3.6 Limpar o código inicial

1. No Explorer, clique na seta ao lado da pasta `lib`.
2. Clique em `main.dart`.
3. Pressione `Ctrl+A` dentro do editor.
4. Pressione Delete e salve com `Ctrl+S`.
5. Não execute ainda: o arquivo vazio gera erro até criarmos o novo conteúdo.

3.7 Estrutura final

```
lib/
■■■ config/
■   ■■■ api_config.dart
■■■ models/
■   ■■■ cliente.dart
■   ■■■ produto.dart
■   ■■■ servico.dart
■   ■■■ venda.dart
■■■ pages/
■   ■■■ home_page.dart
■   ■■■ clientes_page.dart
■   ■■■ produtos_page.dart
■   ■■■ servicos_page.dart
■   ■■■ vendas_page.dart
■■■ services/
■   ■■■ api_service.dart
■■■ widgets/
■   ■■■ ui_helpers.dart
■■■ app.dart
■■■ main.dart
```

Como criar uma pasta ou arquivo no VS Code

1. Passe o mouse sobre a pasta `lib`.
2. Use o ícone **New Folder** para criar uma pasta.
3. Selecione a pasta desejada e use o ícone **New File**.
4. Digite o nome completo, incluindo `.dart`, e pressione Enter.
5. Cole ou digite o código e salve com `Ctrl+S`.

4. Dependências e organização

4.1 Editar o pubspec.yaml

O arquivo `pubspec.yaml` define metadados, versões e bibliotecas. O pacote `http` fará as requisições; `intl` formatará moeda e data.

Arquivo: `pubspec.yaml`

```
name: flutter_gestao_comercial
description: Front-end didático em Flutter para a API Gestão Comercial.
publish_to: "none"
version: 1.0.0+1

environment:
  sdk: ">=3.3.0 <4.0.0"

dependencies:
  flutter:
    sdk: flutter
  http: ^1.2.2
  intl: ^0.19.0
  cupertino_icons: ^1.0.8

dev_dependencies:
  flutter_test:
    sdk: flutter
  flutter_lints: ^5.0.0

flutter:
  uses-material-design: true
```

Depois de salvar, execute:

```
flutter pub get
```

O terminal deve terminar sem mensagens de erro. Se aparecer erro de indentação, lembre-se de que YAML utiliza espaços e não aceita tabulações.

4.2 Criar as pastas

No Explorer do VS Code, clique com o botão direito sobre `lib` e crie as pastas `config`, `models`, `pages`, `services` e `widgets`.

Checkpoint

Execute `flutter analyze`. Neste momento, o projeto ainda pode apontar referências ausentes porque os arquivos serão criados nas próximas etapas. Ao final, o comando deverá retornar `No issues found!`.

5. Configuração do endereço da API

O endereço será centralizado para não precisar procurar URLs em todas as telas.

5.1 Criar o arquivo

1. Expanda a pasta `lib`.
2. Clique com o botão direito em `config`.
3. Selecione **New File**.
4. Digite exatamente `api_config.dart`.
5. Cole o código abaixo e pressione `Ctrl+S`.

Arquivo: `lib/config/api_config.dart`

```
class ApiConfig {
  // Para Flutter Web local: http://localhost:8080/api
  // Para emulador Android: http://10.0.2.2:8080/api
  // Para o Render: https://seu-servico.onrender.com/api
  // Também é possível informar em tempo de execução:
  // flutter run -d chrome --dart-define=API_URL=https://.../api
  static const String baseUrl = String.fromEnvironment(
    'API_URL',
    defaultValue: 'http://localhost:8080/api',
  );
}
```

5.2 Ler o código em linguagem comum

1. `class ApiConfig` cria um agrupador chamado ApiConfig.
2. `static` permite acessar a URL sem criar um objeto.
3. `const` informa que o valor não mudará durante a execução.
4. `String.fromEnvironment` procura o parâmetro API_URL.
5. `defaultValue` é usado quando nenhum parâmetro é informado.

5.3 Escolher a URL

Ambiente	URL
Flutter Web local	http://localhost:8080/api
Emulador Android	http://10.0.2.2:8080/api
API no Render	https://seu-servico.onrender.com/api

Não inclua uma barra no final, pois o serviço acrescentará caminhos como `/clientes`.

Checkpoint

1. Salve o arquivo.
2. Observe se a aba perdeu o círculo branco que indica alteração não salva.
3. Abra **View** → **Problems**.
4. O arquivo api_config.dart não deve apresentar erro.
5. Ainda não tente executar, pois faltam os demais arquivos.

6. Modelagem dos dados

Os modelos representam no Dart as estruturas JSON enviadas e recebidas. Cada classe possui propriedades, construtor, `fromJson` e, quando necessário, `toJson`.

Procedimento para cada modelo

1. Clique com o botão direito na pasta `models`.
2. Crie o arquivo com o nome indicado.
3. Cole o código completo.
4. Salve com Ctrl+S.
5. Leia a explicação imediatamente após o código.
6. Somente então crie o próximo modelo.

6.1 Modelo Cliente

Arquivo: `lib/models/cliente.dart`

```
class Cliente {
  final int? id;
  final String nome;
  final String documento;
  final String? email;
  final String? telefone;
  final String? endereco;
  final bool ativo;

  const Cliente({this.id, required this.nome, required this.documento, this.email, this.telefone,
    this.endereco, this.ativo = true});

  // Converte o JSON recebido da API em um objeto Dart.
  factory Cliente.fromJson(Map<String, dynamic> json) => Cliente(
        id: json['id'] as int?, nome: json['nome'] ?? '', documento: json['documento'] ?? '',
        email: json['email'], telefone: json['telefone'], endereco: json['endereco'], ativo:
          json['ativo'] ?? true,
      );

  // Converte o objeto em JSON para POST e PUT.
  Map<String, dynamic> toJson() => {'nome': nome, 'documento': documento, 'email': email, 'telefone':
    telefone, 'endereco': endereco, 'ativo': ativo};
}
```

fromJson: cria um objeto Dart a partir do mapa recebido. **toJson:** cria um mapa que será serializado no corpo de POST ou PUT.

Exemplo de transformação

```
JSON recebido: {"id": 1, "nome": "Ana", "ativo": true}
Objeto criado: Cliente(id: 1, nome: "Ana", ativo: true)
```

O ID é `int?` porque é nulo antes do cadastro e passa a existir depois que a API salva o cliente.

6.2 Modelo Produto

Arquivo: `lib/models/produto.dart`

```
class Produto {
  final int? id;
  final String nome;
  final String? descricao;
  final double preco;
  final int estoque;
  final bool ativo;

  const Produto({this.id, required this.nome, this.descricao, required this.preco, required this.estoque,
     this.ativo = true});

  factory Produto.fromJson(Map<String, dynamic> json) => Produto(
        id: json['id'] as int?, nome: json['nome'] ?? '', descricao: json['descricao'],
        preco: (json['preco'] as num).toDouble(), estoque: json['estoque'] ?? 0, ativo: json['ativo'] ??
          true,
      );

  Map<String, dynamic> toJson() => {'nome': nome, 'descricao': descricao, 'preco': preco, 'estoque':
    estoque, 'ativo': ativo};
}
```

O JSON pode representar números como inteiro ou decimal. Por isso, o preço é lido como `num` e convertido com `toDouble()`.

Teste mental

Se a API devolver `"preco": 59`, Dart recebe um inteiro. A chamada `toDouble()` produz 59.0 e mantém a propriedade com tipo consistente.

6.3 Modelo Serviço

Arquivo: `lib/models/servico.dart`

```
class Servico {
  final int? id;
  final String nome;
  final String? descricao;
  final double preco;
  final bool ativo;

  const Servico({this.id, required this.nome, this.descricao, required this.preco, this.ativo = true});

  factory Servico.fromJson(Map<String, dynamic> json) => Servico(
        id: json['id'] as int?, nome: json['nome'] ?? '', descricao: json['descricao'],
        preco: (json['preco'] as num).toDouble(), ativo: json['ativo'] ?? true,
      );

  Map<String, dynamic> toJson() => {'nome': nome, 'descricao': descricao, 'preco': preco, 'ativo':
    ativo};
}
```

Compare Serviço e Produto: ambos possuem preço, mas apenas Produto possui estoque. Essa diferença será refletida nos formulários.

6.4 Modelos Venda e ItemVenda

Arquivo: `lib/models/venda.dart`

```
class ItemVenda {
  final int? id;
  final String tipo;
  final int referenciaId;
  final String descricao;
  final int quantidade;
  final double valorUnitario;
  final double subtotal;

  const ItemVenda({this.id, required this.tipo, required this.referenciaId, required this.descricao,
    required this.quantidade, required this.valorUnitario, required this.subtotal});

  factory ItemVenda.fromJson(Map<String, dynamic> json) => ItemVenda(
        id: json['id'], tipo: json['tipo'] ?? '', referenciaId: json['referenciaId'] ?? 0,
        descricao: json['descricao'] ?? '', quantidade: json['quantidade'] ?? 0,
        valorUnitario: (json['valorUnitario'] as num).toDouble(), subtotal: (json['subtotal'] as
          num).toDouble(),
      );
}

class Venda {
  final int id;
  final ClienteResumo cliente;
  final DateTime dataHora;
  final String status;
  final double valorTotal;
  final List<ItemVenda> itens;

  const Venda({required this.id, required this.cliente, required this.dataHora, required this.status,
    required this.valorTotal, required this.itens});

  factory Venda.fromJson(Map<String, dynamic> json) => Venda(
        id: json['id'], cliente: ClienteResumo.fromJson(json['cliente']), dataHora:
          DateTime.parse(json['dataHora']),
        status: json['status'], valorTotal: (json['valorTotal'] as num).toDouble(),
        itens: (json['itens'] as List).map((e) => ItemVenda.fromJson(e)).toList(),
      );
}

class ClienteResumo {
  final int id;
  final String nome;
  const ClienteResumo({required this.id, required this.nome});
  factory ClienteResumo.fromJson(Map<String, dynamic> json) => ClienteResumo(id: json['id'], nome:
    json['nome']);
}
```

Uma venda contém outro objeto, o cliente, e uma lista de itens. `map(...).toList()` percorre o array JSON e cria um ItemVenda para cada elemento.

Checkpoint dos modelos

1. Abra **Terminal** → **New Terminal**.
2. Execute `dart format lib/models`.
3. Execute `flutter analyze`.
4. Se aparecer erro em uma chave JSON, confira aspas, colchetes, vírgulas e parênteses.
5. Continue somente quando os quatro arquivos não apresentarem sublinhado vermelho.

7. Camada de comunicação HTTP

O arquivo `ApiService` concentra todas as requisições. Isso evita que cada tela repita URL, cabeçalhos, serialização e tratamento de erro.

7.1 Criar o serviço

1. Na pasta `services`, crie `api_service.dart`.
2. Cole o código completo apresentado a seguir.
3. Salve o arquivo.
4. Se `package:http/http.dart` ficar sublinhado, abra o terminal e execute `flutter pub get`.
5. Não altere nomes de métodos: as telas os chamarão exatamente como estão escritos.

Arquivo: `lib/services/api_service.dart`

```
import 'dart:convert';
import 'package:http/http.dart' as http;
import '../config/api_config.dart';
import '../models/cliente.dart';
import '../models/produto.dart';
import '../models/servico.dart';
import '../models/venda.dart';

class ApiException implements Exception {
  final String mensagem;
  const ApiException(this.mensagem);
  @override
  String toString() => mensagem;
}

class ApiService {
  final http.Client _client;
  ApiService({http.Client? client}) : _client = client ?? http.Client();

  static const _headers = {'Content-Type': 'application/json; charset=UTF-8'};

  // Interpreta a resposta e transforma erros HTTP em mensagens úteis para a tela.
  dynamic _processar(http.Response response) {
    final body = response.body.isEmpty ? null : jsonDecode(utf8.decode(response.bodyBytes));
    if (response.statusCode >= 200 && response.statusCode < 300) return body;
    final mensagem = body is Map ? body['mensagem'] : null;
    throw ApiException(mensagem?.toString() ?? 'Erro HTTP ${response.statusCode}.');
  }

  Future<List<Cliente>> listarClientes() async {
    final response = await _client.get(Uri.parse('${ApiConfig.baseUrl}/clientes'));
    return (_processar(response) as List).map((e) => Cliente.fromJson(e)).toList();
  }

  Future<Cliente> salvarCliente(Cliente cliente) async {
    final uri = cliente.id == null ? '${ApiConfig.baseUrl}/clientes' :
      '${ApiConfig.baseUrl}/clientes/${cliente.id}';
    final response = cliente.id == null
        ? await _client.post(Uri.parse(uri), headers: _headers, body: jsonEncode(cliente.toJson()))
        : await _client.put(Uri.parse(uri), headers: _headers, body: jsonEncode(cliente.toJson()));
    return Cliente.fromJson(_processar(response));
  }

  Future<void> excluirCliente(int id) async {
    _processar(await _client.delete(Uri.parse('${ApiConfig.baseUrl}/clientes/$id')));
  }

  Future<List<Produto>> listarProdutos() async {
    final response = await _client.get(Uri.parse('${ApiConfig.baseUrl}/produtos'));
    return (_processar(response) as List).map((e) => Produto.fromJson(e)).toList();
  }

  Future<Produto> salvarProduto(Produto produto) async {
    final uri = produto.id == null ? '${ApiConfig.baseUrl}/produtos' :
```

```
      '${ApiConfig.baseUrl}/produtos/${produto.id}';
    final response = produto.id == null
        ? await _client.post(Uri.parse(uri), headers: _headers, body: jsonEncode(produto.toJson()))
        : await _client.put(Uri.parse(uri), headers: _headers, body: jsonEncode(produto.toJson()));
    return Produto.fromJson(_processar(response));
  }

  Future<void> excluirProduto(int id) async {
    _processar(await _client.delete(Uri.parse('${ApiConfig.baseUrl}/produtos/$id')));
  }

  Future<List<Servico>> listarServicos() async {
    final response = await _client.get(Uri.parse('${ApiConfig.baseUrl}/servicos'));
    return (_processar(response) as List).map((e) => Servico.fromJson(e)).toList();
  }

  Future<Servico> salvarServico(Servico servico) async {
    final uri = servico.id == null ? '${ApiConfig.baseUrl}/servicos' :
      '${ApiConfig.baseUrl}/servicos/${servico.id}';
    final response = servico.id == null
        ? await _client.post(Uri.parse(uri), headers: _headers, body: jsonEncode(servico.toJson()))
        : await _client.put(Uri.parse(uri), headers: _headers, body: jsonEncode(servico.toJson()));
    return Servico.fromJson(_processar(response));
  }

  Future<void> excluirServico(int id) async {
    _processar(await _client.delete(Uri.parse('${ApiConfig.baseUrl}/servicos/$id')));
  }

  Future<List<Venda>> listarVendas() async {
    final response = await _client.get(Uri.parse('${ApiConfig.baseUrl}/vendas'));
    return (_processar(response) as List).map((e) => Venda.fromJson(e)).toList();
  }

  Future<Venda> criarVenda({required int clienteId, required List<Map<String, dynamic>> itens}) async {
    final response = await _client.post(Uri.parse('${ApiConfig.baseUrl}/vendas'), headers: _headers,
      body: jsonEncode({'clienteId': clienteId, 'itens': itens}));
    return Venda.fromJson(_processar(response));
  }

  Future<Venda> cancelarVenda(int id) async {
    final response = await _client.patch(Uri.parse('${ApiConfig.baseUrl}/vendas/$id/cancelamento'));
    return Venda.fromJson(_processar(response));
  }
}
```

7.2 Entender os imports

```
import 'dart:convert';
import 'package:http/http.dart' as http;
import '../config/api_config.dart';
```

- `dart:convert` fornece jsonEncode e jsonDecode.
- `http` envia requisições pela rede.
- `../` significa voltar uma pasta para localizar config e models.

7.3 Anatomia de uma requisição GET

```
final response = await _client.get(
  Uri.parse('${ApiConfig.baseUrl}/clientes'),
);
```

1. `Uri.parse` transforma o texto em endereço.
2. `get` envia a requisição HTTP.
3. `await` aguarda sem congelar a interface.
4. `_processar` valida o status e decodifica o JSON.
5. `map` converte a lista JSON em objetos Cliente.

7.4 POST e PUT

Quando `id == null`, o registro ainda não existe e o método usado é POST. Quando há ID, o serviço executa PUT.

7.5 DELETE e Future

DELETE bem-sucedido normalmente retorna status 204 e corpo vazio. Por isso, o método não precisa devolver um objeto.

7.6 Status HTTP

Status	Significado para o estudante
200/201/204	Operação realizada
400	Campo inválido ou ausente
404	ID não encontrado
409	Documento ou e-mail duplicado
422	Regra de negócio: estoque, inativo etc.
500	Erro do back-end; consultar logs

Checkpoint

```
dart format lib/services/api_service.dart
flutter analyze
```

Se o analisador apontar “Target of URI does not exist: package:http”, o `pubspec.yaml` não foi salvo ou `flutter pub get` ainda não foi executado.

8. Widgets reutilizáveis

Crie componentes para carregamento, lista vazia, mensagens e confirmação. Assim, as telas mantêm comportamento consistente.

Passo a passo

1. Clique na pasta `widgets`.
2. Crie `ui_helpers.dart`.
3. Cole o código e salve.
4. Use `Shift+Alt+F` para formatar no Windows.
5. Execute `flutter analyze`.

Arquivo: `lib/widgets/ui_helpers.dart`

```
import 'package:flutter/material.dart';

class LoadingView extends StatelessWidget {
  const LoadingView({super.key});
  @override
  Widget build(BuildContext context) => const Center(child: CircularProgressIndicator());
}

class EmptyView extends StatelessWidget {
  final String mensagem;
  const EmptyView(this.mensagem, {super.key});
  @override
  Widget build(BuildContext context) => Center(child: Column(mainAxisSize: MainAxisSize.min, children:
    [const Icon(Icons.inbox_outlined, size: 64), const SizedBox(height: 12), Text(mensagem)]));
}

void mostrarMensagem(BuildContext context, String mensagem, {bool erro = false}) {
  ScaffoldMessenger.of(context).showSnackBar(SnackBar(content: Text(mensagem), backgroundColor: erro ?
    Theme.of(context).colorScheme.error : null));
}

Future<bool> confirmarExclusao(BuildContext context, String nome) async {
  return await showDialog<bool>(
        context: context,
        builder: (context) => AlertDialog(
          title: const Text('Confirmar exclusão'),
          content: Text('Deseja realmente excluir “$nome”?'),
          actions: [TextButton(onPressed: () => Navigator.pop(context, false), child: const
            Text('Cancelar')), FilledButton(onPressed: () => Navigator.pop(context, true), child: const
            Text('Excluir'))],
        ),
      ) ?? false;
}
```

Conceitos aplicados

- **LoadingView:** aparece enquanto a Future não terminou.
- **EmptyView:** orienta quando a API devolve lista vazia.
- **mostrarMensagem:** apresenta sucesso ou erro.
- **confirmarExclusao:** impede exclusões acidentais.
- **BuildContext:** permite localizar tema, navegador e mensagens na árvore de widgets.

Não se preocupe se esses componentes ainda não aparecerem. Eles somente serão usados depois que as páginas forem criadas.

9. Aplicação, tema e navegação

9.1 Ponto de entrada

Arquivo: `lib/main.dart`

```
import 'package:flutter/material.dart';
import 'app.dart';

// Ponto de entrada da aplicação Flutter.
void main() {
  runApp(const GestaoComercialApp());
}
```

A função `main()` é o primeiro código executado. `runApp` coloca o widget principal na árvore.

9.2 MaterialApp e tema

Arquivo: `lib/app.dart`

```
import 'package:flutter/material.dart';
import 'pages/home_page.dart';

class GestaoComercialApp extends StatelessWidget {
  const GestaoComercialApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'Gestão Comercial',
      debugShowCheckedModeBanner: false,
      theme: ThemeData(
        colorScheme: ColorScheme.fromSeed(seedColor: const Color(0xFF1E5AA8)),
        useMaterial3: true,
        inputDecorationTheme: const InputDecorationTheme(
          border: OutlineInputBorder(),
          filled: true,
        ),
        cardTheme: const CardThemeData(elevation: 1, margin: EdgeInsets.all(8)),
      ),
      home: const HomePage(),
    );
  }
}
```

O `MaterialApp` configura título, tema, modo Material 3 e tela inicial.

9.3 Navegação responsiva

Arquivo: `lib/pages/home_page.dart`

```
import 'package:flutter/material.dart';
import 'clientes_page.dart';
import 'produtos_page.dart';
import 'servicos_page.dart';
import 'vendas_page.dart';

class HomePage extends StatefulWidget {
  const HomePage({super.key});
  @override
  State<HomePage> createState() => _HomePageState();
}

class _HomePageState extends State<HomePage> {
  int _indice = 0;
  final _paginas = const [ClientesPage(), ProdutosPage(), ServicosPage(), VendasPage()];
  final _titulos = const ['Clientes', 'Produtos', 'Serviços', 'Vendas'];
  final _destinos = const [
    NavigationDestination(icon: Icon(Icons.people_outline), selectedIcon: Icon(Icons.people), label:
      'Clientes'),
    NavigationDestination(icon: Icon(Icons.inventory_2_outlined), selectedIcon: Icon(Icons.inventory_2),
      label: 'Produtos'),
    NavigationDestination(icon: Icon(Icons.build_outlined), selectedIcon: Icon(Icons.build), label:
      'Serviços'),
    NavigationDestination(icon: Icon(Icons.point_of_sale_outlined), selectedIcon:
      Icon(Icons.point_of_sale), label: 'Vendas'),
  ];

  @override
  Widget build(BuildContext context) {
    final largura = MediaQuery.sizeOf(context).width;
    final desktop = largura >= 800;
    return Scaffold(
      appBar: AppBar(title: Text('Gestão Comercial • ${_titulos[_indice]}')),
      body: Row(children: [
        if (desktop)
          NavigationRail(
            selectedIndex: _indice,
            onDestinationSelected: (valor) => setState(() => _indice = valor),
            labelType: NavigationRailLabelType.all,
            destinations: _destinos.map((d) => NavigationRailDestination(icon: d.icon, selectedIcon:
              d.selectedIcon, label: Text(d.label))).toList(),
          ),
        Expanded(child: IndexedStack(index: _indice, children: _paginas)),
      ]),
      bottomNavigationBar: desktop ? null : NavigationBar(selectedIndex: _indice, onDestinationSelected:
        (valor) => setState(() => _indice = valor), destinations: _destinos),
    );
  }
}
```

Em telas com 800 pixels ou mais, a navegação aparece lateralmente com `NavigationRail`. Em telas menores, utiliza `NavigationBar` na parte inferior.

Por que IndexedStack?

O `IndexedStack` mantém as páginas já construídas, preservando estado e evitando recriação completa ao alternar abas.

10. CRUD de clientes

O módulo de clientes será o primeiro CRUD completo. Ele reúne GET, POST, PUT, DELETE, formulário, validação, diálogo e atualização da lista.

Arquivo: `lib/pages/clientes_page.dart`

```
import 'package:flutter/material.dart';
import '../models/cliente.dart';
import '../services/api_service.dart';
import '../widgets/ui_helpers.dart';

class ClientesPage extends StatefulWidget {
  const ClientesPage({super.key});
  @override
  State<ClientesPage> createState() => _ClientesPageState();
}

class _ClientesPageState extends State<ClientesPage> {
  final _api = ApiService();
  List<Cliente> _clientes = [];
  bool _carregando = true;

  @override
  void initState() { super.initState(); _carregar(); }

  Future<void> _carregar() async {
    setState(() => _carregando = true);
    try { _clientes = await _api.listarClientes(); }
    catch (e) { if (mounted) mostrarMensagem(context, e.toString(), erro: true); }
    finally { if (mounted) setState(() => _carregando = false); }
  }

  Future<void> _abrirFormulario([Cliente? cliente]) async {
    final formKey = GlobalKey<FormState>();
    final nome = TextEditingController(text: cliente?.nome);
    final documento = TextEditingController(text: cliente?.documento);
    final email = TextEditingController(text: cliente?.email);
    final telefone = TextEditingController(text: cliente?.telefone);
    final endereco = TextEditingController(text: cliente?.endereco);
    bool ativo = cliente?.ativo ?? true;

    final salvo = await showDialog<bool>(context: context, builder: (context) => StatefulBuilder(builder:
       (context, setDialogState) => AlertDialog(
      title: Text(cliente == null ? 'Novo cliente' : 'Editar cliente'),
      content: SizedBox(width: 520, child: Form(key: formKey, child: SingleChildScrollView(child:
        Column(mainAxisSize: MainAxisSize.min, children: [
        TextFormField(controller: nome, decoration: const InputDecoration(labelText: 'Nome'), validator:
          (v) => v == null || v.trim().isEmpty ? 'Informe o nome.' : null),
        const SizedBox(height: 12), TextFormField(controller: documento, decoration: const
          InputDecoration(labelText: 'CPF/CNPJ'), validator: (v) => v == null || v.trim().isEmpty ?
          'Informe o documento.' : null),
        const SizedBox(height: 12), TextFormField(controller: email, decoration: const
          InputDecoration(labelText: 'E-mail'), keyboardType: TextInputType.emailAddress),
        const SizedBox(height: 12), TextFormField(controller: telefone, decoration: const
          InputDecoration(labelText: 'Telefone')),
        const SizedBox(height: 12), TextFormField(controller: endereco, decoration: const
          InputDecoration(labelText: 'Endereço')),
        SwitchListTile(value: ativo, onChanged: (v) => setDialogState(() => ativo = v), title: const
          Text('Cliente ativo')),
      ]))),
      actions: [TextButton(onPressed: () => Navigator.pop(context, false), child: const
        Text('Cancelar')), FilledButton(onPressed: () async {
        if (!formKey.currentState!.validate()) return;
        try {
          await _api.salvarCliente(Cliente(id: cliente?.id, nome: nome.text.trim(), documento:
            documento.text.trim(), email: email.text.trim().isEmpty ? null : email.text.trim(), telefone:
             telefone.text.trim(), endereco: endereco.text.trim(), ativo: ativo));
          if (context.mounted) Navigator.pop(context, true);
        } catch (e) { if (context.mounted) mostrarMensagem(context, e.toString(), erro: true); }
      }, child: const Text('Salvar'))],
    )));
    if (salvo == true) { await _carregar(); if (mounted) mostrarMensagem(context, 'Cliente salvo com
      sucesso.'); }
  }
```

```
  Future<void> _excluir(Cliente cliente) async {
    if (!await confirmarExclusao(context, cliente.nome)) return;
    try { await _api.excluirCliente(cliente.id!); await _carregar(); if (mounted)
      mostrarMensagem(context, 'Cliente excluído.'); }
    catch (e) { if (mounted) mostrarMensagem(context, e.toString(), erro: true); }
  }

  @override
  Widget build(BuildContext context) => Scaffold(
    body: _carregando ? const LoadingView() : RefreshIndicator(
      onRefresh: _carregar,
      child: _clientes.isEmpty ? const EmptyView('Nenhum cliente cadastrado.') : ListView.builder(
        padding: const EdgeInsets.all(12), itemCount: _clientes.length,
        itemBuilder: (_, i) { final c = _clientes[i]; return Card(child: ListTile(
          leading: CircleAvatar(child: Text(c.nome.isEmpty ? '?' : c.nome[0].toUpperCase())),
          title: Text(c.nome), subtitle: Text('${c.documento} • ${c.email ?? 'sem e-mail'}'),
          trailing: Row(mainAxisSize: MainAxisSize.min, children: [Icon(c.ativo ? Icons.check_circle :
            Icons.block, color: c.ativo ? Colors.green : Colors.grey), IconButton(onPressed: () =>
            _abrirFormulario(c), icon: const Icon(Icons.edit)), IconButton(onPressed: () => _excluir(c),
            icon: const Icon(Icons.delete_outline))]),
        )); },
      ),
    ),
    floatingActionButton: FloatingActionButton.extended(onPressed: () => _abrirFormulario(), icon: const
      Icon(Icons.add), label: const Text('Novo cliente')),
  );
}
```

10.1 Ciclo da tela

1. `initState` chama `_carregar()`.
2. A variável `_carregando` exibe o indicador.
3. A API devolve a lista e `setState` reconstrói a tela.
4. O botão flutuante abre o formulário sem cliente, produzindo POST.
5. O botão editar abre o formulário com os dados, produzindo PUT.
6. O botão excluir confirma e executa DELETE.

10.2 Formulário e validação

O `GlobalKey<FormState>` permite executar todas as validações com `formKey.currentState!.validate()`. O salvamento só continua se os validadores retornarem `null`.

Teste orientado

1. Execute a API e o Flutter.
2. Abra Clientes.
3. Cadastre um cliente com nome, documento e e-mail.
4. Edite o telefone do registro.
5. Atualize a página e confirme a persistência.
6. Tente repetir o documento e observe o erro 409.
7. Exclua um cliente que ainda não possui vendas.

11. CRUD de produtos

Produtos acrescentam campos numéricos e estoque.

Arquivo: `lib/pages/produtos_page.dart`

```
import 'package:flutter/material.dart';
import 'package:intl/intl.dart';
import '../models/produto.dart';
import '../services/api_service.dart';
import '../widgets/ui_helpers.dart';

class ProdutosPage extends StatefulWidget {
  const ProdutosPage({super.key});
  @override State<ProdutosPage> createState() => _ProdutosPageState();
}

class _ProdutosPageState extends State<ProdutosPage> {
  final _api = ApiService();
  final _moeda = NumberFormat.currency(locale: 'pt_BR', symbol: 'R\$');
  List<Produto> _itens = [];
  bool _carregando = true;
  @override void initState() { super.initState(); _carregar(); }

  Future<void> _carregar() async { setState(() => _carregando = true); try { _itens = await
    _api.listarProdutos(); } catch (e) { if (mounted) mostrarMensagem(context, e.toString(), erro: true);
     } finally { if (mounted) setState(() => _carregando = false); } }

  Future<void> _formulario([Produto? produto]) async {
    final key = GlobalKey<FormState>();
    final nome = TextEditingController(text: produto?.nome);
    final descricao = TextEditingController(text: produto?.descricao);
    final preco = TextEditingController(text: produto?.preco.toStringAsFixed(2).replaceAll('.', ','));
    final estoque = TextEditingController(text: produto?.estoque.toString() ?? '0');
    bool ativo = produto?.ativo ?? true;
    final salvo = await showDialog<bool>(context: context, builder: (context) => StatefulBuilder(builder:
       (context, setLocal) => AlertDialog(
      title: Text(produto == null ? 'Novo produto' : 'Editar produto'),
      content: SizedBox(width: 500, child: Form(key: key, child: SingleChildScrollView(child:
        Column(mainAxisSize: MainAxisSize.min, children: [
        TextFormField(controller: nome, decoration: const InputDecoration(labelText: 'Nome'), validator:
          _obrigatorio), const SizedBox(height: 12),
        TextFormField(controller: descricao, decoration: const InputDecoration(labelText: 'Descrição'),
          maxLines: 2), const SizedBox(height: 12),
        TextFormField(controller: preco, decoration: const InputDecoration(labelText: 'Preço'),
          keyboardType: const TextInputType.numberWithOptions(decimal: true), validator:
          _numeroPositivo), const SizedBox(height: 12),
        TextFormField(controller: estoque, decoration: const InputDecoration(labelText: 'Estoque'),
          keyboardType: TextInputType.number, validator: _inteiroNaoNegativo),
        SwitchListTile(value: ativo, onChanged: (v) => setLocal(() => ativo = v), title: const
          Text('Produto ativo')),
      ]))),
      actions: [TextButton(onPressed: () => Navigator.pop(context, false), child: const
        Text('Cancelar')), FilledButton(onPressed: () async {
        if (!key.currentState!.validate()) return;
        try { await _api.salvarProduto(Produto(id: produto?.id, nome: nome.text.trim(), descricao:
          descricao.text.trim(), preco: double.parse(preco.text.replaceAll(',', '.')), estoque:
          int.parse(estoque.text), ativo: ativo)); if (context.mounted) Navigator.pop(context, true); }
        catch (e) { if (context.mounted) mostrarMensagem(context, e.toString(), erro: true); }
      }, child: const Text('Salvar'))],
    )));
    if (salvo == true) { await _carregar(); if (mounted) mostrarMensagem(context, 'Produto salvo.'); }
  }

  String? _obrigatorio(String? v) => v == null || v.trim().isEmpty ? 'Campo obrigatório.' : null;
  String? _numeroPositivo(String? v) { final n = double.tryParse((v ?? '').replaceAll(',', '.')); return
    n == null || n < 0 ? 'Informe um valor válido.' : null; }
  String? _inteiroNaoNegativo(String? v) { final n = int.tryParse(v ?? ''); return n == null || n < 0 ?
    'Informe um número inteiro válido.' : null; }
  Future<void> _excluir(Produto p) async { if (!await confirmarExclusao(context, p.nome)) return; try {
    await _api.excluirProduto(p.id!); await _carregar(); } catch (e) { if (mounted)
    mostrarMensagem(context, e.toString(), erro: true); } }

  @override Widget build(BuildContext context) => Scaffold(
    body: _carregando ? const LoadingView() : _itens.isEmpty ? const EmptyView('Nenhum produto
      cadastrado.') : RefreshIndicator(onRefresh: _carregar, child: ListView.builder(padding: const
      EdgeInsets.all(12), itemCount: _itens.length, itemBuilder: (_, i) { final p = _itens[i]; return
```

```
      Card(child: ListTile(
      leading: const CircleAvatar(child: Icon(Icons.inventory_2)), title: Text(p.nome), subtitle:
        Text('${_moeda.format(p.preco)} • Estoque: ${p.estoque}'),
      trailing: Row(mainAxisSize: MainAxisSize.min, children: [IconButton(onPressed: () =>
        _formulario(p), icon: const Icon(Icons.edit)), IconButton(onPressed: () => _excluir(p), icon:
        const Icon(Icons.delete_outline))]),
    )); })),
    floatingActionButton: FloatingActionButton.extended(onPressed: () => _formulario(), icon: const
      Icon(Icons.add), label: const Text('Novo produto')),
  );
}
```

11.1 Conversão decimal

O usuário brasileiro costuma digitar vírgula. Antes de usar `double.parse`, o código troca vírgula por ponto:

```
double.parse(preco.text.replaceAll(',', '.'))
```

11.2 Regras da interface

- Preço deve ser um número maior ou igual a zero.
- Estoque deve ser inteiro e não negativo.
- O indicador de carregamento aparece durante a busca.
- Pull-to-refresh executa nova consulta.
- Exclusão pode falhar se o produto estiver referenciado em vendas.

Teste orientado

1. Cadastre dois produtos com estoques diferentes.
2. Edite o preço de um produto.
3. Confirme os valores formatados em reais.
4. Registre uma venda posteriormente e confira a redução do estoque.

12. CRUD de serviços

Serviços são semelhantes a produtos, porém não possuem estoque.

Arquivo: `lib/pages/servicos_page.dart`

```
import 'package:flutter/material.dart';
import 'package:intl/intl.dart';
import '../models/servico.dart';
import '../services/api_service.dart';
import '../widgets/ui_helpers.dart';

class ServicosPage extends StatefulWidget {
  const ServicosPage({super.key});
  @override State<ServicosPage> createState() => _ServicosPageState();
}

class _ServicosPageState extends State<ServicosPage> {
  final _api = ApiService();
  final _moeda = NumberFormat.currency(locale: 'pt_BR', symbol: 'R\$');
  List<Servico> _itens = [];
  bool _carregando = true;
  @override void initState() { super.initState(); _carregar(); }
  Future<void> _carregar() async { setState(() => _carregando = true); try { _itens = await
    _api.listarServicos(); } catch (e) { if (mounted) mostrarMensagem(context, e.toString(), erro: true);
     } finally { if (mounted) setState(() => _carregando = false); } }

  Future<void> _formulario([Servico? servico]) async {
    final key = GlobalKey<FormState>();
    final nome = TextEditingController(text: servico?.nome);
    final descricao = TextEditingController(text: servico?.descricao);
    final preco = TextEditingController(text: servico?.preco.toStringAsFixed(2).replaceAll('.', ','));
    bool ativo = servico?.ativo ?? true;
    final salvo = await showDialog<bool>(context: context, builder: (context) => StatefulBuilder(builder:
       (context, setLocal) => AlertDialog(
      title: Text(servico == null ? 'Novo serviço' : 'Editar serviço'),
      content: SizedBox(width: 500, child: Form(key: key, child: Column(mainAxisSize: MainAxisSize.min,
        children: [
        TextFormField(controller: nome, decoration: const InputDecoration(labelText: 'Nome'), validator:
          (v) => v == null || v.trim().isEmpty ? 'Informe o nome.' : null), const SizedBox(height: 12),
        TextFormField(controller: descricao, decoration: const InputDecoration(labelText: 'Descrição'),
          maxLines: 2), const SizedBox(height: 12),
        TextFormField(controller: preco, decoration: const InputDecoration(labelText: 'Preço'),
          keyboardType: const TextInputType.numberWithOptions(decimal: true), validator: (v) { final n =
          double.tryParse((v ?? '').replaceAll(',', '.')); return n == null || n < 0 ? 'Informe um preço
          válido.' : null; }),
        SwitchListTile(value: ativo, onChanged: (v) => setLocal(() => ativo = v), title: const
          Text('Serviço ativo')),
      ]))),
      actions: [TextButton(onPressed: () => Navigator.pop(context, false), child: const
        Text('Cancelar')), FilledButton(onPressed: () async {
        if (!key.currentState!.validate()) return;
        try { await _api.salvarServico(Servico(id: servico?.id, nome: nome.text.trim(), descricao:
          descricao.text.trim(), preco: double.parse(preco.text.replaceAll(',', '.')), ativo: ativo)); if
           (context.mounted) Navigator.pop(context, true); }
        catch (e) { if (context.mounted) mostrarMensagem(context, e.toString(), erro: true); }
      }, child: const Text('Salvar'))],
    )));
    if (salvo == true) { await _carregar(); if (mounted) mostrarMensagem(context, 'Serviço salvo.'); }
  }

  Future<void> _excluir(Servico s) async { if (!await confirmarExclusao(context, s.nome)) return; try {
    await _api.excluirServico(s.id!); await _carregar(); } catch (e) { if (mounted)
    mostrarMensagem(context, e.toString(), erro: true); } }

  @override Widget build(BuildContext context) => Scaffold(
    body: _carregando ? const LoadingView() : _itens.isEmpty ? const EmptyView('Nenhum serviço
      cadastrado.') : RefreshIndicator(onRefresh: _carregar, child: ListView.builder(padding: const
      EdgeInsets.all(12), itemCount: _itens.length, itemBuilder: (_, i) { final s = _itens[i]; return
      Card(child: ListTile(
      leading: const CircleAvatar(child: Icon(Icons.build)), title: Text(s.nome), subtitle:
        Text('${_moeda.format(s.preco)} • ${s.ativo ? 'Ativo' : 'Inativo'}'),
      trailing: Row(mainAxisSize: MainAxisSize.min, children: [IconButton(onPressed: () =>
        _formulario(s), icon: const Icon(Icons.edit)), IconButton(onPressed: () => _excluir(s), icon:
        const Icon(Icons.delete_outline))]),
    )); })),
    floatingActionButton: FloatingActionButton.extended(onPressed: () => _formulario(), icon: const
```

```
        Icon(Icons.add), label: const Text('Novo serviço')),
    );
  }
```

Comparação

Característica	Produto	Serviço
Preço	Sim	Sim
Descrição	Sim	Sim
Ativo	Sim	Sim
Estoque	Sim	Não
Baixa na venda	Reduz quantidade	Não altera estoque

A semelhança entre as telas é proposital para facilitar a aprendizagem. Em um projeto maior, poderíamos extrair formulários e listas genéricas, mas isso acrescentaria abstração antes de os alunos dominarem o fluxo básico.

13. Vendas: integração dos módulos

A tela de vendas combina dados de clientes, produtos e serviços. Ela demonstra requisições paralelas, seleção de recursos, lista temporária de itens e envio de uma estrutura JSON composta.

Arquivo: `lib/pages/vendas_page.dart`

```
import 'package:flutter/material.dart';
import 'package:intl/intl.dart';
import '../models/cliente.dart';
import '../models/produto.dart';
import '../models/servico.dart';
import '../models/venda.dart';
import '../services/api_service.dart';
import '../widgets/ui_helpers.dart';

class VendasPage extends StatefulWidget {
  const VendasPage({super.key});
  @override State<VendasPage> createState() => _VendasPageState();
}

class _VendasPageState extends State<VendasPage> {
  final _api = ApiService();
  final _moeda = NumberFormat.currency(locale: 'pt_BR', symbol: 'R\$');
  final _data = DateFormat('dd/MM/yyyy HH:mm');
  List<Venda> _vendas = [];
  bool _carregando = true;
  @override void initState() { super.initState(); _carregar(); }
  Future<void> _carregar() async { setState(() => _carregando = true); try { _vendas = await
    _api.listarVendas(); } catch (e) { if (mounted) mostrarMensagem(context, e.toString(), erro: true); }
     finally { if (mounted) setState(() => _carregando = false); } }

  Future<void> _novaVenda() async {
    try {
      final resultados = await Future.wait([_api.listarClientes(), _api.listarProdutos(),
        _api.listarServicos()]);
      final clientes = resultados[0] as List<Cliente>;
      final produtos = resultados[1] as List<Produto>;
      final servicos = resultados[2] as List<Servico>;
      if (!mounted) return;
      if (clientes.isEmpty || (produtos.isEmpty && servicos.isEmpty)) { mostrarMensagem(context,
        'Cadastre um cliente e pelo menos um produto ou serviço.', erro: true); return; }

      int? clienteId = clientes.first.id;
      String tipo = produtos.isNotEmpty ? 'PRODUTO' : 'SERVICO';
      int? referenciaId = produtos.isNotEmpty ? produtos.first.id : servicos.first.id;
      final quantidade = TextEditingController(text: '1');
      final itens = <Map<String, dynamic>>[];

      final criada = await showDialog<bool>(context: context, builder: (context) =>
        StatefulBuilder(builder: (context, setLocal) {
        final List<dynamic> opcoes = tipo == 'PRODUTO' ? produtos : servicos;
        return AlertDialog(
          title: const Text('Nova venda'),
          content: SizedBox(width: 600, child: SingleChildScrollView(child: Column(mainAxisSize:
            MainAxisSize.min, children: [
            DropdownButtonFormField<int>(initialValue: clienteId, decoration: const
              InputDecoration(labelText: 'Cliente'), items: clientes.where((c) => c.ativo).map((c) =>
              DropdownMenuItem(value: c.id, child: Text(c.nome))).toList(), onChanged: (v) => setLocal(()
               => clienteId = v)),
            const SizedBox(height: 16), SegmentedButton<String>(segments: const [ButtonSegment(value:
              'PRODUTO', label: Text('Produto'), icon: Icon(Icons.inventory_2)), ButtonSegment(value:
              'SERVICO', label: Text('Serviço'), icon: Icon(Icons.build))], selected: {tipo},
              onSelectionChanged: (v) => setLocal(() { tipo = v.first; final lista = tipo == 'PRODUTO' ?
              produtos : servicos; referenciaId = lista.isEmpty ? null : lista.first.id; })),
            const SizedBox(height: 16),
            if (opcoes.isNotEmpty) DropdownButtonFormField<int>(key: ValueKey(tipo), initialValue:
              referenciaId, decoration: InputDecoration(labelText: tipo == 'PRODUTO' ? 'Produto' :
              'Serviço'), items: opcoes.map((item) => DropdownMenuItem<int>(value: item.id, child:
              Text(item.nome))).toList(), onChanged: (v) => setLocal(() => referenciaId = v))
            else const Text('Nenhum item deste tipo cadastrado.'),
            const SizedBox(height: 12), TextFormField(controller: quantidade, decoration: const
              InputDecoration(labelText: 'Quantidade'), keyboardType: TextInputType.number),
            const SizedBox(height: 12), OutlinedButton.icon(onPressed: referenciaId == null ? null : () {
               final q = int.tryParse(quantidade.text); if (q == null || q < 1) return; setLocal(() =>
              itens.add({'tipo': tipo, 'referenciaId': referenciaId, 'quantidade': q})); }, icon: const
```

```
              Icon(Icons.add_shopping_cart), label: const Text('Adicionar item')),
            const Divider(height: 28), ...itens.asMap().entries.map((e) => ListTile(dense: true, title:
              Text('${e.value['tipo']} #${e.value['referenciaId']}'), subtitle: Text('Quantidade:
              ${e.value['quantidade']}'), trailing: IconButton(icon: const Icon(Icons.close), onPressed:
              () => setLocal(() => itens.removeAt(e.key)))),
          ])),
          actions: [TextButton(onPressed: () => Navigator.pop(context, false), child: const
            Text('Cancelar')), FilledButton(onPressed: itens.isEmpty ? null : () async { try { await
            _api.criarVenda(clienteId: clienteId!, itens: itens); if (context.mounted)
            Navigator.pop(context, true); } catch (e) { if (context.mounted) mostrarMensagem(context,
            e.toString(), erro: true); } }, child: const Text('Concluir venda'))],
        );
      }));
      if (criada == true) { await _carregar(); if (mounted) mostrarMensagem(context, 'Venda registrada
        com sucesso.'); }
    } catch (e) { if (mounted) mostrarMensagem(context, e.toString(), erro: true); }
  }

  Future<void> _cancelar(Venda venda) async { try { await _api.cancelarVenda(venda.id); await
    _carregar(); if (mounted) mostrarMensagem(context, 'Venda cancelada e estoque devolvido.'); } catch
    (e) { if (mounted) mostrarMensagem(context, e.toString(), erro: true); } }

  @override Widget build(BuildContext context) => Scaffold(
    body: _carregando ? const LoadingView() : _vendas.isEmpty ? const EmptyView('Nenhuma venda
      registrada.') : RefreshIndicator(onRefresh: _carregar, child: ListView.builder(padding: const
      EdgeInsets.all(12), itemCount: _vendas.length, itemBuilder: (_, i) { final v = _vendas[i]; final
      cancelada = v.status == 'CANCELADA'; return Card(child: ExpansionTile(
      leading: CircleAvatar(child: Text('#${v.id}')), title: Text(v.cliente.nome), subtitle:
        Text('${_data.format(v.dataHora.toLocal())} • ${_moeda.format(v.valorTotal)}'),
      trailing: Chip(label: Text(v.status), backgroundColor: cancelada ? Colors.red.shade100 :
        Colors.green.shade100),
      children: [...v.itens.map((item) => ListTile(dense: true, title: Text(item.descricao), subtitle:
        Text('${item.quantidade} × ${_moeda.format(item.valorUnitario)}'), trailing:
        Text(_moeda.format(item.subtotal)))), if (!cancelada) Padding(padding: const EdgeInsets.all(12),
        child: Align(alignment: Alignment.centerRight, child: OutlinedButton.icon(onPressed: () =>
        _cancelar(v), icon: const Icon(Icons.cancel_outlined), label: const Text('Cancelar venda'))))],
    )); })),
    floatingActionButton: FloatingActionButton.extended(onPressed: _novaVenda, icon: const
      Icon(Icons.add), label: const Text('Nova venda')),
  );
}
```

13.1 Carregamento paralelo

```
final resultados = await Future.wait([
  _api.listarClientes(),
  _api.listarProdutos(),
  _api.listarServicos(),
]);
```

`Future.wait` aguarda as três consultas. Isso reduz o tempo total em comparação com esperar uma requisição terminar para iniciar a próxima.

13.2 Estrutura enviada

```
{
  "clienteId": 1,
  "itens": [
    {"tipo": "PRODUTO", "referenciaId": 1, "quantidade": 2},
    {"tipo": "SERVICO", "referenciaId": 1, "quantidade": 1}
  ]
}
```

O front-end não envia preço nem total. A API consulta os preços oficiais, verifica estoque, calcula subtotais, reduz produtos e registra a venda.

13.3 Cancelamento

O método PATCH altera o estado da venda para CANCELADA. No back-end, os produtos são devolvidos ao estoque. A interface recarrega a listagem para refletir o novo estado.

Teste integrado

1. Garanta que exista cliente ativo, produto com estoque e serviço ativo.
2. Crie uma venda com dois produtos e um serviço.
3. Expanda o cartão da venda.
4. Confira quantidade, preço unitário, subtotal e total.
5. Abra Produtos e confirme a redução do estoque.
6. Cancele a venda.
7. Volte a Produtos e confirme a devolução.

14. Execução e integração com Render

14.1 Executar com API local

```
flutter clean
flutter pub get
flutter run -d chrome
```

O endereço padrão será `http://localhost:8080/api`.

14.2 Executar usando a API hospedada

```
flutter run -d chrome --dart-define=API_URL=https://SEU-SERVICO.onrender.com/api
```

Substitua o domínio pelo endereço verdadeiro e mantenha `/api` no final.

14.3 Gerar a versão web

```
flutter build web --release --dart-define=API_URL=https://SEU-SERVICO.onrender.com/api
```

Os arquivos finais serão criados em:

```
build/web/
```

14.4 Hot reload e hot restart

Comando	Função
r	Hot reload: atualiza mudanças preservando estado
R	Hot restart: reinicia a aplicação
q	Encerra a execução

Alterações em constantes definidas com `--dart-define` exigem encerrar e executar novamente.

15. Depuração e problemas comuns

Sintoma	Causa provável	Correção
Failed host lookup	URL incorreta ou sem internet	Revisar API_URL e conexão
XMLHttpRequest error	API fora do ar ou CORS	Abrir a URL no navegador e verificar CORS
Connection refused	Back-end local não iniciado	Executar o Spring Boot
FormatException	Resposta não é JSON esperado	Conferir Network e Swagger
409 Conflict	Documento/e-mail duplicado	Usar dados únicos
422 Unprocessable Entity	Regra de negócio	Ler a mensagem devolvida
500 Internal Server Error	Falha no back-end	Consultar logs do Render
Dropdown sem itens	Cadastros básicos ausentes	Cadastrar cliente/produto/serviço

15.1 Ferramentas do Chrome

1. Com o aplicativo aberto, pressione `F12`.
2. Abra a aba **Network**.
3. Realize uma operação no Flutter.
4. Selecione a requisição para visualizar URL, método, status, payload e resposta.
5. Use a aba Console para mensagens de execução.

15.2 Comandos de recuperação

```
flutter clean
flutter pub get
flutter analyze
flutter run -d chrome
```

15.3 Checklist final

- A API abre pelo navegador.
- `flutter doctor -v` reconhece Chrome e VS Code.
- `flutter pub get` termina sem erro.
- `flutter analyze` não indica problemas.
- A URL possui protocolo e termina em `/api`.
- Clientes, produtos e serviços permitem cadastrar e editar.
- Venda calcula total pelo back-end e reduz estoque.
- Cancelamento devolve o estoque.
- Erros são apresentados ao usuário.

Roteiro didático sugerido

Momento	Tempo	Atividade
1	15 min	Revisão da arquitetura e demonstração da API
2	20 min	Criação do projeto, dependências e pastas
3	25 min	Configuração, modelos e conversão JSON
4	30 min	ApiService e testes GET/POST
5	35 min	Tela de clientes construída em conjunto
6	30 min	Produtos e serviços em duplas
7	35 min	Integração da tela de vendas
8	20 min	Depuração, Render e fechamento

Desafio de consolidação

Implemente uma busca por nome nas telas de clientes, produtos e serviços. A API já aceita o parâmetro `?nome=`. Inclua um `TextField`, envie o valor ao serviço e atualize a lista.

Desafio avançado

Crie uma tela inicial com indicadores: quantidade de clientes, total de produtos, produtos com estoque baixo e total de vendas concluídas. Inicialmente, os valores podem ser calculados a partir das listas. Depois, discuta a vantagem de criar um endpoint específico de dashboard.

Critérios de avaliação

Critério	Peso
Organização em pastas e legibilidade	20%
Consumo correto da API	25%
Formulários e validações	20%
Tratamento de estados e erros	15%
Venda integrada e estoque	15%
Apresentação e explicação do código	5%

Roteiro autônomo de execução

Use esta lista como controle. Marque uma etapa somente quando o resultado esperado tiver sido observado.

#	Ação	Resultado esperado
1	Executar flutter doctor -v	Flutter, Chrome e VS Code reconhecidos
2	Abrir o Swagger da API	Página carrega e GET retorna 200
3	Criar e executar projeto padrão	Contador aparece no Chrome
4	Adicionar http e intl	flutter pub get sem erro
5	Criar pastas e ApiConfig	Arquivo sem sublinhado vermelho
6	Criar quatro modelos	flutter analyze sem erros nesses arquivos
7	Criar ApiService	Pacote http reconhecido
8	Criar ui_helpers	Widgets reconhecidos
9	Criar main, app e home	Aplicação abre com quatro abas
10	Criar ClientesPage	Lista e cadastro funcionam
11	Criar ProdutosPage	Preço e estoque aparecem
12	Criar ServicosPage	Cadastro funciona
13	Criar VendasPage	Venda com itens é registrada
14	Cancelar venda	Status muda e estoque retorna
15	Executar com API_URL do Render	Dados remotos aparecem

Quando parar e revisar

- Se houver sublinhado vermelho em um import, não continue criando outras telas.
- Se flutter analyze mostrar erro, resolva o primeiro erro antes dos demais.
- Se a API não responder no Swagger, não altere o Flutter para tentar compensar.
- Se a tela ficar vermelha, leia a primeira linha da mensagem e anote arquivo e linha.
- Se a interface abrir mas não listar, verifique Network no Chrome.

Como pedir ajuda de forma eficiente

Mesmo sendo autoinstrucional, problemas de ambiente podem ocorrer. Ao solicitar ajuda, envie: comando executado, primeira mensagem de erro, arquivo e linha, URL utilizada e print da aba Network. “Não funcionou” não fornece dados suficientes para diagnóstico.

Referência rápida de comandos

```
flutter doctor -v
flutter devices
flutter create flutter_gestao_comercial
flutter pub get
dart format lib
flutter analyze
flutter run -d chrome
flutter clean
flutter run -d chrome --dart-define=API_URL=https://SEU-SERVICO.onrender.com/api
flutter build web --release --dart-define=API_URL=https://SEU-SERVICO.onrender.com/api
```

Glossário final

Termo	Definição curta
Build	Processo de gerar a aplicação
Context	Posição do widget na árvore
Endpoint	Endereço de uma operação da API
Estado	Dado que pode alterar a interface
Hot reload	Atualização rápida preservando estado
Import	Inclusão de código de outro arquivo/pacote
Payload	Dados enviados no corpo da requisição
Repository/back-end	Camada que acessa o banco; não existe no Flutter
Serialização	Objeto para JSON
Desserialização	JSON para objeto
Widget tree	Hierarquia dos componentes visuais

Conclusão

Ao concluir a apostila, mesmo partindo sem experiência em Flutter, o estudante terá desenvolvido um cliente completo para uma API REST e compreendido a função de cada camada. A sequência também prepara para evoluções como autenticação, Provider, Riverpod, Dio, paginação e testes automatizados.